

Runtime Adaptivity for CSE and Machine Learning with Charm4Py, CharmTyles, and CharmNumerics

Ritvik Rao, Aditya Bhosale, Laxmikant Kale

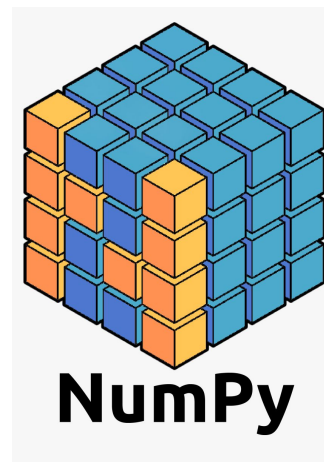
Traditional HPC

- Scientific computing
 - Large scientific simulations
 - Heavy on linear algebra
- Programming environment
 - Language: C++
 - Programming models: MPI, Charm++, etc.
 - Hardware: HPC clusters with CPUs/GPUs/nodes connected with Infiniband

NAMD
Scalable Molecular Dynamics

New large-scale workloads

- Machine learning
- Data analysis
- Different prog environment
 - Language: Python and Python-based libraries
 - Programming models: Automated/abstracted parallelism using Python libraries
 - Hardware: Increasing use of cloud computing

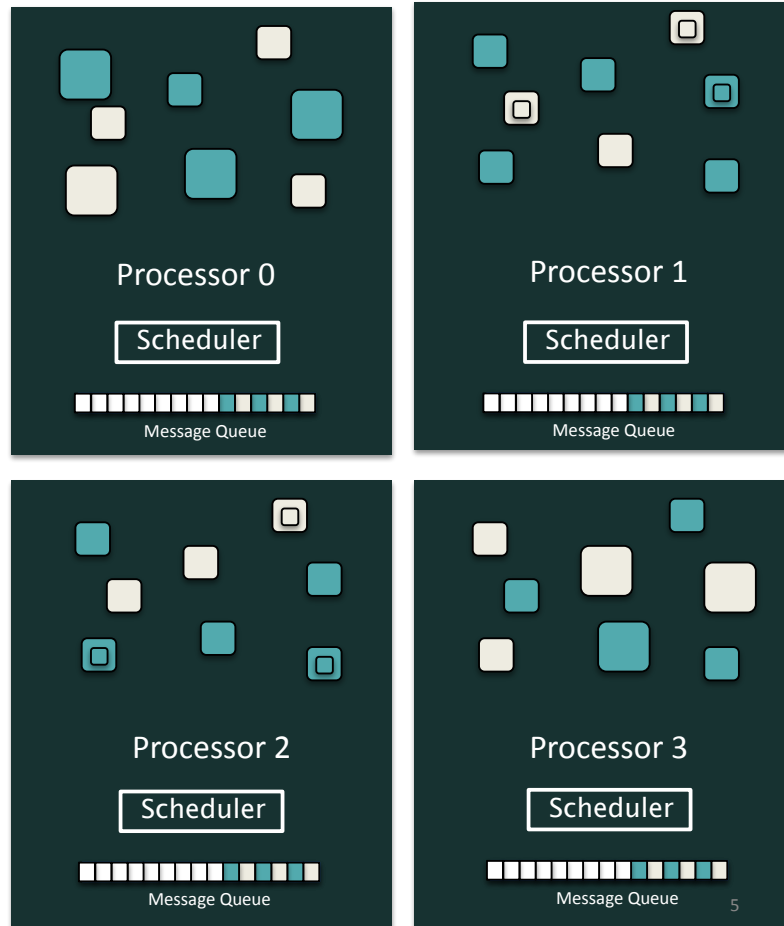


Need to bridge the two

- Future of computing defined by AI/newer applications
 - More Python, more cloud
- Traditional applications need to adapt
- Also use concepts from HPC to improve new workloads
 - i.e. dynamic load balancing for AI inference
- Our solution: **a new suite of programming models and libraries based on Charm++**
 - Charm4Py
 - CharmTyles
 - CharmNumerics

Charm++

- C++ runtime system based on migratable objects
- Dynamic load balancing, computation-communication overlap
- Proven for traditional HPC applications
 - Molecular dynamics
 - Cosmological simulations



Charm4Py

- Python implementation of Charm++
- Uses Charm++ runtime underneath
 - Interfaces with C++ with Cython
- Designed for traditional HPC applications that use Python
 - E.g. scientific applications that process data with Pandas
- Charm++ provides load balancing/communication, Python used for local computation
- Charm4Py can be used with other similar runtimes
- Documentation: <https://charm4py.readthedocs.io/en/latest/>

Charm4Py Hello world

```
from charm4py import charm, Chare

class Hello(Chare):
    def __init__(self):
        print("Hello from PE: " + str(charm.myPe()))
        exit()

def main(args):
    c = Chare(Hello, onPe=0)

charm.start(main)
```

Ray on Charm4Py

- Ray: Python programming model that uses the actor model
- Conceptually similar to Charm4Py but with inefficiencies
 - Centralized task scheduler
 - User-estimated resource usage (must provide resource requirement for each object)
- Idea: Implement Ray abstractions with Charm4Py
- Reuse existing Ray programs, except for replacing “import ray” with “from charm4py import ray”
- Represent Ray objects as Charm4Py chares, use Charm++ schedulers and load balancing

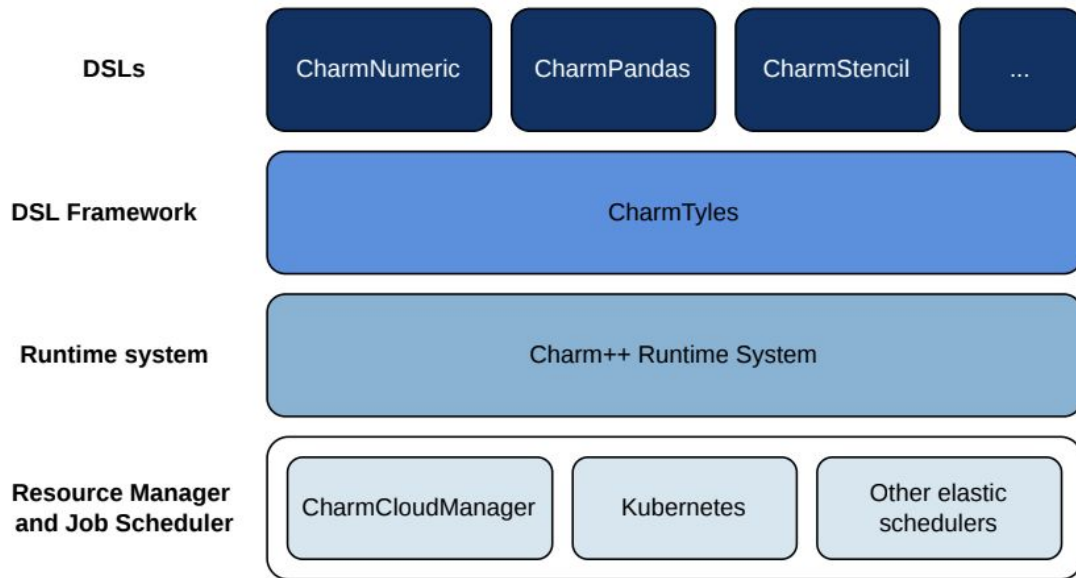
Charm on the cloud

- Need to take advantage of:
 - Heterogeneous resources
 - Low-cost options like spot instances
- Charm++ operator in Kubernetes
- Shrink-expand of resources during execution
 - Charm++ load balancing to move resources off of lost spot instance/on to gained instance
- Charm++ and Kokkos to execute across clusters of different types of GPUs
- CharmTyles for abstracting frontend from backend hardware execution

CharmTypes

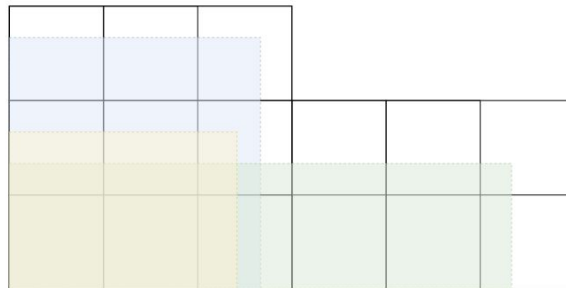
- Interactive computing with Python
- Allow Python code to call Charm++ libraries
- Communicate between Python frontend and Charm++ backend with AST
- Basis for many useful DSLs
 - CharmStencil
 - CharmPandas
 - CharmNumerics

CharmTyles ecosystem



CharmNumerics

- Num4Py abstractions implemented in parallel with Charm++
- Designed to run on CPUs and GPUs
- Intended to support dense linear algebra applications
- Supports various features such as custom slicing and tiling



Future/need for collaboration

- Charm4Py:
 - Existing developers: Port applications to Charm4Py
 - Implement intra-process parallelism and load balancing
 - If you are interested: **Charm4Py tutorial at IPDPS next week, Monday & Tuesday 8:30-12:00**
- CharmTyles:
 - Expansion of libraries/DSLs that are based on CharmTyles
- CharmNumerics:
 - Large scientific/data applications
- All of these need applications built on top of them

References

J. J. Galvez, K. Senthil, and L. Kale, “Charmpy: A python parallel programming model,” in *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, 2018, pp. 423–433.

A. Bhosale and L. Kale, “An Adaptive Distributed Stencil Abstraction for GPUs,” in *2025 Supercomputing India (SCI)*, 2025, pp. 1–8.

A. Bhosale, K. Chandrasekar, L. Kale, and S. Kokkila-Schumacher, “An elastic job scheduler for HPC applications on the cloud,” in *Proceedings of the SC’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 151–162.

A. Bhosale, L. Kale, and S. Kokkila-Schumacher, “Efficient and Cost-Effective HPC on the Cloud,” in *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*, 2025, pp. 1–5.